

Project Report

AgenticAI Project Trio

Automated AI Video Synthesis Pipeline

Built by

Hussain Ali Zaidi (22i-0902)

Hamza Ahmed (22i-1339)

Muhammad Daniyal Aziz (22i-0753)

Saifullah Khalid (22i-1312)

May 5, 2026

Abstract

This report outlines the design, implementation, and architectural evaluation of an automated, end-to-end AI video synthesis pipeline. Structured as a five-phase modular system, the project leverages Large Language Models (LLMs) via LangGraph, text-to-speech synthesis, image generation, and a robust REST/WebSocket-based web server. The architecture is bounded by a singular, tightly typed JSON schema contract using Pydantic, ensuring invariant enforcement across asynchronous agent workflows. The deployment stack uses Python 3.11.9, Uvicorn, and FastAPI to deliver a high-throughput microservice architecture. The pipeline integrates configurable visual backends (remote SDXL, local SDXL, OpenAI Images, and a deterministic placeholder renderer), a lip-sync layer (Wav2Lip with a passthrough fallback), and a compositing layer powered by FFmpeg. An edit-agent subsystem provides idempotent, append-only history and natural language driven video editing by re-running only the necessary phases. The result is a resilient framework capable of converting abstract prompts into complete, audio-synced video shorts with deterministic fallback strategies.

Project Collaborators

Built by Hussain Ali Zaidi (22i-0902), Hamza Ahmed (22i-1339), Muhammad Daniyal Aziz (22i-0753), Saifullah Khalid (22i-1312).

Individual Contributions

- **Hussain Ali Zaidi:** Led Phase 1 (LangGraph story generation) and Phase 2 (audio and deterministic timing). Implemented multi-agent structure, validation tools, emotion tagging, and Edge-TTS integration.
- **Hamza Ahmed:** Led Phase 3 (image and video generation). Integrated image backend chain, SVD-based motion generation, Wav2Lip lip-sync client, and FFmpeg compositing logic.
- **Muhammad Daniyal Aziz:** Led Phase 4 (web API and orchestration). Implemented FastAPI routes, WebSocket progress bus, and pipeline orchestration with persistent project directories.
- **Saifullah Khalid:** Led Phase 5 (editing agent and versioning). Implemented edit intent classifier, edit executor, and append-only snapshot versioning.

Chapter 1

Introduction

As the demand for rapid multimedia content generation grows, automating the video creation pipeline represents a central challenge in Agentic AI. The AgenticAI Project Trio provides an end-to-end, multi-agent pipeline that converts a single text prompt into a complete video with narration, background music, subtitles, and versioned edits. The system is intentionally modular and phase-driven, preventing cascading failures and ensuring reproducible artifacts.

The design is anchored by three principles:

- **Single source of truth:** All phases read and write a single Pydantic model, `ProjectState`, which acts as a validated JSON contract.
- **Deterministic fallbacks:** Every external dependency (LLM, TTS, image generation, lip-sync) has a safe fallback path so the pipeline still completes.
- **Append-only history:** Every change is snapshotted as an immutable version, enabling non-destructive edits and reverts.

Chapter 2

System Architecture

The system architecture prioritizes robust boundaries, typed data flow, and minimal coupling between phases. Each phase is a well-defined transformation of `ProjectState` and writes its outputs to disk under a project directory.

2.1 Pipeline Overview

Prompt → Phase 1 (story) → Phase 2 (audio) → Phase 3 (video) → Phase 4 (API) → Phase 5 (edits).

2.2 The Single Contract: ProjectState

Everything flows through one central Pydantic schema: `ProjectState`. Phase 1 populates story and characters, Phase 2 adds timing data, Phase 3 adds scene video metadata and final output path, Phase 4 exposes the model through the API, and Phase 5 mutates only the necessary subtrees based on edit intent.

Listing 2.1: Core Domain Schema Structure

```
1 class ProjectState(BaseModel):
2     project_id: str
3     version: int = 1
4     phase: PipelinePhase
5     story: Optional[Story] = None
6     timing: Optional[TimingManifest] = None
7     scene_videos: list[SceneVideo] = Field(default_factory=list)
```

By implementing `model_validate()` at every phase boundary, integration bugs are caught and localized early. Each phase updates `ProjectState` and persists `state.json` in the project directory.

2.3 Append-Only State Management

Storage and version control dynamically leverage the filesystem as an append-only audit log. The `StateManager` copies the `current/` directory to `vN/` on each snapshot and logs metadata to `history.json`. A revert is implemented by copying a historical snapshot into `current/` and

recording the action as a new version. This preserves full auditability and supports unlimited undo.

2.4 Project Directory Layout

Projects are stored under `outputs/projects/<project_id>` with a `current/` working directory and versioned snapshots `v1/`, `v2/`, etc.

Chapter 3

Phase-Wise Implementation

The infrastructure separates computational loads efficiently: reasoning, audio structuring, frame generation, serving, and mutational updates. Each phase has a clear API and a strict input/output contract.

3.1 Phase 1: Story Generation (LangGraph)

Phase 1 constructs the narrative skeleton using a sequential LangGraph pipeline with three agents. The implementation is in `phase1_story/agent.py` and tools are in `phase1_story/tools.py`.

- **Story agent:** Generates a `StoryOutline` (title, arc, scene outlines) using `STORY_OUTLINE_PROMPT` or `stub_outline` if no API key is present.
- **Character agent:** Generates a `CharacterRoster` and validates it with `check_consistency` (roles, unique names, voice styles).
- **Script agent:** Produces `ScriptOutput` with full scenes, dialogue lines, and visual prompts.

Validation tools enforce structural and semantic rules: `validate_story_arc`, `estimate_duration`, `build_visual_prompt`, and `analyze_emotions`. Each agent retries once and falls back to deterministic stubs on repeated validation failure.

If the LLM is unavailable, `schemas/llm.py` provides deterministic stubs (`stub_outline`, `stub_roster`, `stub_script`, `stub_story`) so the rest of the pipeline can proceed.

3.2 Phase 2: Audio Synthesis and Deterministic Timing

Phase 2 is implemented in `phase2_audio/pipeline.py` and produces a `TimingManifest`. It resolves voices for each character, synthesizes dialogue lines, and computes a global timeline using a monotonic `cursor_ms`. Each audio file is stored under `audio/` and referenced by a relative path in the manifest.

Key behaviors:

- **Voice resolution:** `voice_for` maps each character to a TTS voice and fills `Character.voice_id`.
- **Emotion-aware prosody:** The optional handoff file `phase2_audio/handoff.json` contains emotion tags (from Phase 1), which are mapped to `prosody_for` settings (rate and pitch).

- **Deterministic alignment:** Each `AudioSegment` contains `start_ms` and `end_ms`, enabling frame-perfect alignment in Phase 3.
- **Scene padding:** Each scene is padded to `duration_seconds` so visuals and audio remain synchronized.
- **BGM generation:** `pick_bgm` writes per-scene background music files, recorded as `kind="bgm"` segments.

The helper `realign_scene_durations` updates scene lengths directly from the manifest to ensure the video hold time matches audio playback.

Audio files are validated for size before use. If a synthesized MP3 is missing or too small, the pipeline falls back to a WAV file and still records it in the manifest. This keeps the timeline consistent even under degraded conditions.

3.3 Phase 3: Image and Video Synthesis

Phase 3 is implemented in `phase3_video/pipeline.py`. It converts the `TimingManifest` and scene metadata into per-line clips, per-scene MP4s, and the final video. `compositor.compose_scene` concatenates dialogue clips, mixes BGM, and burns subtitles using FFmpeg `drawtext`. Final assembly uses `compositor.concat_with_transitions` with crossfades and a hard-cut fallback.

3.3.1 Image Generation Backends

Scene and speaker portraits are generated by `phase3_video/image_gen.py` using a backend chain (remote SDXL, local SDXL, OpenAI Images, placeholder PIL). Both `generate_scene_image` and `generate_speaker_image` share the same chain, guaranteeing a usable image even without external services.

3.3.2 Motion and Lip Sync

Per dialogue line, `run_phase3` generates a speaker portrait, optionally produces a motion clip via SVD, and lip-syncs using Wav2Lip when available. If the remote endpoint is missing, the system uses a passthrough mux that preserves timing. The Wav2Lip client ensures audio is resampled to 16 kHz mono WAV using FFmpeg and will always return a valid MP4. This guarantees the compositing stage can proceed even when advanced models are unavailable.

3.4 Phase 4: Web Layer and Orchestration

Phase 4 provides the user-facing API in `phase4_web/api.py` and the orchestrator in `phase4_web/orchestrator`. Uvicorn runs the ASGI server and delivers REST and WebSocket endpoints.

3.4.1 API Specifications and Endpoints

The HTTP server initializes bound to `127.0.0.1:8000`. Key endpoints include:

- `GET /`: Transmits core application HTML components.

- **GET /api/projects & /api/health:** Retrieves existing pipeline operations and hardware availability indicators, returning HTTP 200 OK structures.
- **POST /api/projects:** Boots pipeline execution, yielding asynchronous project IDs.
- **GET /api/projects/{id}:** Submits the compiled JSON `ProjectState` map.
- **WS /api/projects/{id}/ws:** Establishes long-polling WebSockets. Handlers accept the upgrade request, subsequently transmitting live inference updates (e.g., SVD or Wav2Lip generation percentages or console progress bars).
- **POST /api/projects/{id}/edit:** Submits prompt modification requests to routing.
- **GET ../file/final_output.mp4:** Serves binary video to the browser using HTTP 206 Partial Content so HTML5 `<video>` tags can seek and buffer.
- **GET /assets/app.js & /assets/styles.css:** Core cacheable (HTTP 304) assets implementing rendering layers.

Progress streaming is implemented via a `ProgressBus` that maps project IDs to per-client queues. Each phase emits messages through a callback so multiple WebSocket clients can observe the same task.

3.5 Phase 5: Edit Agent and Executor

Phase 5 enables natural language edits through a strict `EditIntent` schema and a targeted re-render pipeline. The intent classifier maps queries to `EditIntent`, while the executor reruns only the affected phases and uses `scope_filter` to control regeneration.

3.6 JSON Schema Design

The schema in `schemas/pipeline.py` formalizes every piece of state in the pipeline. Core entities include `Story`, `Scene`, `Character`, `DialogueLine`, `AudioSegment`, `TimingManifest`, `SceneVideo`, and the umbrella `ProjectState` model. Strict validation prevents malformed outputs from crossing phase boundaries.

3.7 Tools and APIs Used

- LangGraph for multi-agent story and edit intent graphs.
- OpenAI Chat (gpt-4o-mini) for JSON-structured generation.
- OpenAI Images (dall-e-3), SDXL (diffusers), and a PIL fallback for stills.
- Edge-TTS for speech synthesis and FFmpeg/ffprobe for muxing and probes.
- Wav2Lip for lip-sync via a remote endpoint with passthrough fallback.
- FastAPI + Uvicorn for REST and WebSocket orchestration.
- Pydantic v2 for schema validation and JSON serialization.

Chapter 4

Evaluation

4.1 Results

The pipeline generates complete video projects under `outputs/projects/<project_id>/current`. Representative frames and the final MP4 are accessible through the web UI and the project folder.

4.2 Challenges Faced

- **Multi-agent consistency:** Phase 1 must ensure character names and scene numbers remain consistent across outline, roster, and script. The validation tools were designed to catch structural and semantic mismatches early.
- **Deterministic timing:** Audio lengths drive visual timing, so the system uses a global `cursor_ms` timeline and pads scenes to their declared durations.
- **Model availability:** Image generation, lip-sync, and LLM calls are optional, so robust fallbacks (stubs, placeholder rendering, audio passthrough) were implemented to keep the pipeline functional offline.
- **Cross-platform video compositing:** Subtitles and transitions must work on diverse machines. FFmpeg-based drawtext and xfade pipelines avoid runtime issues from GUI-dependent rendering tools.

Chapter 5

Conclusions and Future Work

The AgenticAI Project Trio delivers a complete, verifiable, and extensible pipeline for AI-driven short film generation. The architecture combines a strict JSON contract, modular phase boundaries, and fail-safe fallbacks, producing reliable artifacts across local and remote environments.

Future work will focus on richer camera motion, improved character consistency across scenes, and higher fidelity compositing. Additional enhancements include expanded editing intents, multi-speaker scene mixing, and richer subtitle styling presets.